

hermetic_bridge_run.sh — companion guide (§11.4.18)

Revision: 1 **Last modified:** 2026-07-02T00:00:00Z **Status:** H2 for the hermetic WireGuard test harness ([design](#)) — the first concrete §11.4.52 **promotion** of an operator-gated protocol test to autonomous. Authority: inherits constitution/Constitution.md per §11.4.35.

Overview

tests/vpn_lan/hermetic_bridge_run.sh proves the hermetic harness actually **promotes** an operator-gated VPN-LAN protocol test to run **autonomously** (§11.4.52). Inside one unshare -Urm it: (1) stands up the H0-full kernel-WireGuard tunnel between two unprivileged netns; (2) runs a **real peer service** in netns B bound to the WG-only overlay address 10.10.0.2; (3) points the bridge contract (tests/lib/sword_bridge.sh) at that peer; (4) runs the **UNMODIFIED** protocol test *inside* netns A, where its own bridge_require gate flips from honest-SKIP to UP and it produces a **real PASS** with captured evidence — no operator, no podman, no Mullvad.

First promotion: chromecast_dial.sh T6.2 (the eureka_info control leg). A peer HTTP server serves setup/eureka_info (a real {"name": ...} JSON) on 10.10.0.2:8008; the test GETs it over the tunnel and asserts a device name. The other legs (discovery / liveness / reverse callback) SKIP honestly (no reflector / status-cmd / observer), exactly as designed.

How the bridge contract is pointed at the hermetic peer

sword_bridge.sh is driven purely by its six contract variables; the harness sets them to the loopback peer so bridge_require returns UP:

var	hermetic value
HELIX_SWORD_DIR	the served dir (non-empty)
	true

var	hermetic value
HELIX_BRIDGE_CONNECT / _DISCONNECT	
HELIX_BRIDGE_HEALTH	a real TCP probe of 10.10.0.2:8008 over the tunnel (not a rubber-stamp)
HELIX_BRIDGE_SUBNET	10.10.0.0/24
HELIX_BRIDGE_HOST / HELIX_VPN_CAST_IP	10.10.0.2

HELIX_BRIDGE_MODE=hermetic is exported to document intent; the current library has no MODE branch, so the promotion is realised by the contract vars above. A later step may add an explicit hermetic branch to the library (H2 proper).

Usage

```
tests/vpn_lan/hermetic_bridge_run.sh # PASS /
SKIP / FAIL
H2_MUT=badeureka tests/vpn_lan/hermetic_bridge_run.sh # $1.1
golden-bad
```

Evidence: qa-results/vpn_lan/hermetic_bridge/<UTC-ts>_<pass|mut>_<pid>/run.evidence (the promoted test also writes its own qa-results/vpn_lan/phase6/...).

Anti-bluff design

The normal PASS is emitted **only** when the promoted chromecast_dial.sh exits 0 **and** its stdout carries a real ^PASS:.*eureka_info line — a SKIP-only run can never satisfy that. The **golden-bad** (H2_MUT=badeureka) serves a 200 body with **no name field**; the only acceptable outcome is the real test's T6.2 fail-closed branch firing (^FAIL:.*eureka_info, exit ≠ 0). This proves the promotion exercises the genuine assertion, not a rubber-stamp (§11.4.107(10) / §11.4.68).

Self-fetch freshness cross-check (§11.4.107 not-stale).

Before running the promoted test, the harness GETs 10.10.0.2:8008/setup/eureka_info **itself** over the tunnel from netns A and asserts THIS run's fresh name nonce is present (normal) / absent (golden-bad). This ties the eventual PASS to a real encrypted round-trip of *our* data — decoupled from the exact chromecast_dial.sh output string, and forbidding a stale/cached/wrong peer from satisfying the test. A **coupling-contract guard**

(`grep -q 'eureka_info' "$TEST"`) makes a future rename of the promoted test's eureka leg fail with a clear diagnostic here rather than a silent grep miss downstream.

Host-safety self-bounding. The peer HTTP server runs under `timeout -k 2 90` *inside* the netns (direct parent of python3), so an outer-SIGKILL orphan self-terminates — no indefinite linger (§12). The outer `timeout 70` reaps the whole unshare tree first in the pathological case, leaving the inner self-bound as the belt-and-suspenders guard.

Captured evidence (verified 2026-07-02)

Normal run embedded the promoted test's own output:

```
chromecast_dial: sword bridge UP – running live Cast/DIAL checks
(subnet=10.10.0.0/24 host=10.10.0.2)
PASS: Cast eureka_info device-name JSON (routed 8008)
[evidence: .../phase6/.../eureka/eureka.evidence]
H2_PASS: the UNMODIFIED chromecast_dial.sh eureka control leg
produced a REAL PASS over the hermetic WireGuard tunnel
```

3/3 deterministic (§11.4.50). Golden-bad → the real test FAILED on the name-less eureka.

Honest scope (§11.4.6 / §11.4.3)

Proves the **eureka control-leg logic** autonomously over an encrypted tunnel against a controlled peer. It does NOT prove a real Chromecast on the real Mullvad topology (that stays the §11.4.3 operator-gated confirmation), and it does not promote the discovery / liveness / reverse-callback legs (those need a reflector / a live receiver / an ingress observer — future H2.x or operator-gated).

Prerequisites / SKIP

bash, unshare+nsenter, iproute2 ip (wireguard link type), wg, host wireguard kernel module, python3, curl, plus tests/vpn_lan/chromecast_dial.sh. Any missing → honest SKIP: (§11.4.3). Process-headroom guard SKIPS on a starved host (§12).

Security (§11.4.10)

WireGuard private keys: mode-0600 mktemp inside the namespace, used by path, removed on exit, never logged. The served eureka name is a per-run nonce (fresh value, §11.4.107 not-stale).

Related

- [hermetic_wg_roundtrip.md](#) — the H0-full tunnel this reuses.
- [hermetic_netns_poc.md](#) — the veth substrate.
- `tests/vpn_lan/chromecast_dial.sh` — the promoted protocol test.

Last verified

2026-07-02 (host: ALT Linux kernel 6.12, wireguard module, wireguard-tools 1.0.20210914, python3, curl, jq).